

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {

    int pipes[2][2];
    pid_t pid[3];

    /* Create two pipes */
    pipe(pipes[0]);
    pipe(pipes[1]);

    /* First process: ls */
    pid[0] = fork();

    if (pid[0] == 0) {

        /* Send output to first pipe */
        dup2(pipes[0][1], STDOUT_FILENO);

        close(pipes[0][0]);
        close(pipes[0][1]);
        close(pipes[1][0]);
        close(pipes[1][1]);
```

```

        exit(1);
    }

    /* Second process: grep */
    pid[1] = fork();

    if (pid[1] == 0) {

        /* Take input from first pipe */
        dup2(pipes[0][0], STDIN_FILENO);

        /* Send output to second pipe */
        dup2(pipes[1][1], STDOUT_FILENO);

        close(pipes[0][0]);
        close(pipes[0][1]);
        close(pipes[1][0]);
        close(pipes[1][1]);

        execlp("grep", "grep", ".c", NULL);

        exit(1);
    }

    /* Third process: wc */
    pid[2] = fork();

```

```

/* Third process: wc */
pid[2] = fork();

if (pid[2] == 0) {

    /* Take input from second pipe */
    dup2(pipes[1][0], STDIN_FILENO);

    close(pipes[0][0]);
    close(pipes[0][1]);
    close(pipes[1][0]);
    close(pipes[1][1]);

    execlp("wc", "wc", "-l", NULL);

    exit(1);
}

/* Parent closes all pipe descriptors */
close(pipes[0][0]);
close(pipes[0][1]);
close(pipes[1][0]);
close(pipes[1][1]);

/* Wait for all three processes */
waitpid(pid[0], NULL, 0);

```

```

/* Wait for all three processes */
waitpid(pid[0], NULL, 0);
waitpid(pid[1], NULL, 0);
waitpid(pid[2], NULL, 0);

printf("Long pipeline completed successfully.\n");

return 0;

```

Output:

```

geya@LenovoL480:~/OS/Skill/Week 6$ gcc task4.c -o task4
geya@LenovoL480:~/OS/Skill/Week 6$ ./task4
4
Long pipeline completed successfully.
geya@LenovoL480:~/OS/Skill/Week 6$

```